Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

Column-Oriented Database Systems

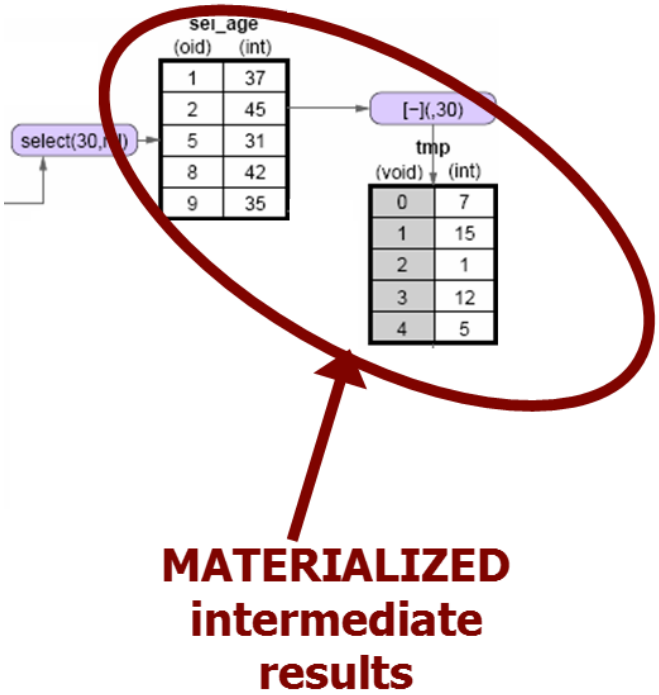
VLDB 2009 Tutorial

vectorized query processing

Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

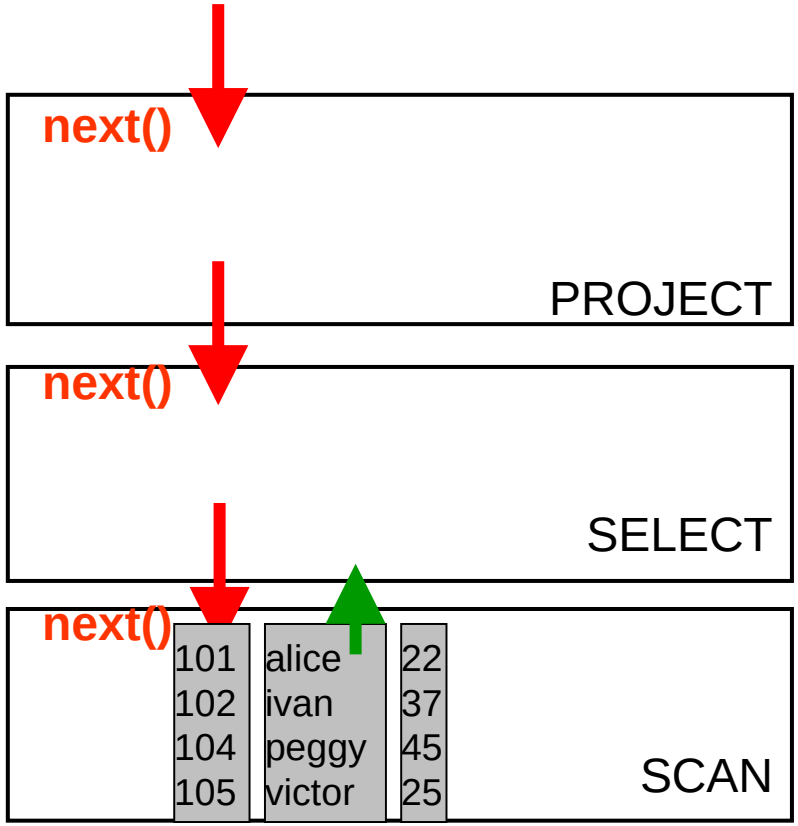
MonetDB spin-off: MonetDB/X100

Materialization vs Pipelining



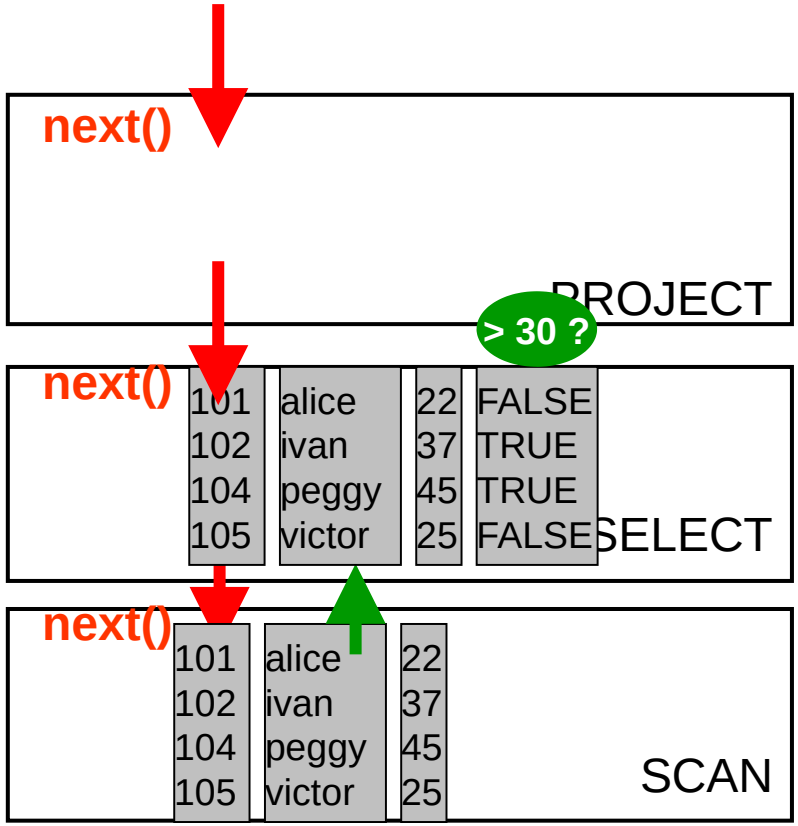
Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

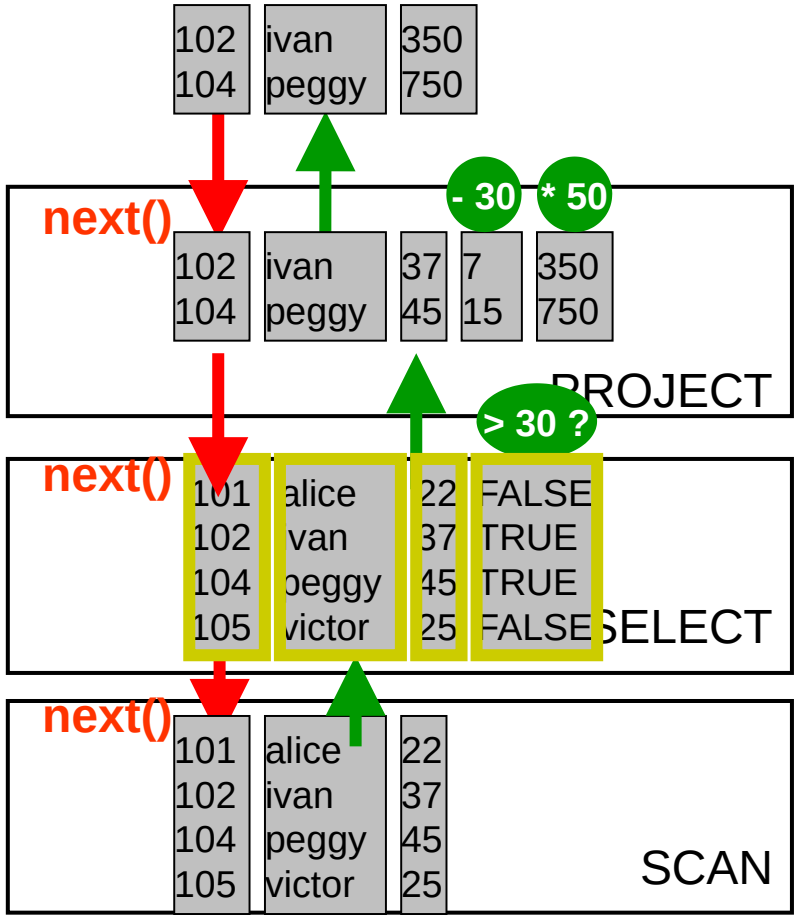
"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

"Vectorized In Cache Processing"

vector = array of ~100

processed in a tight loop

CPU cache Resident



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

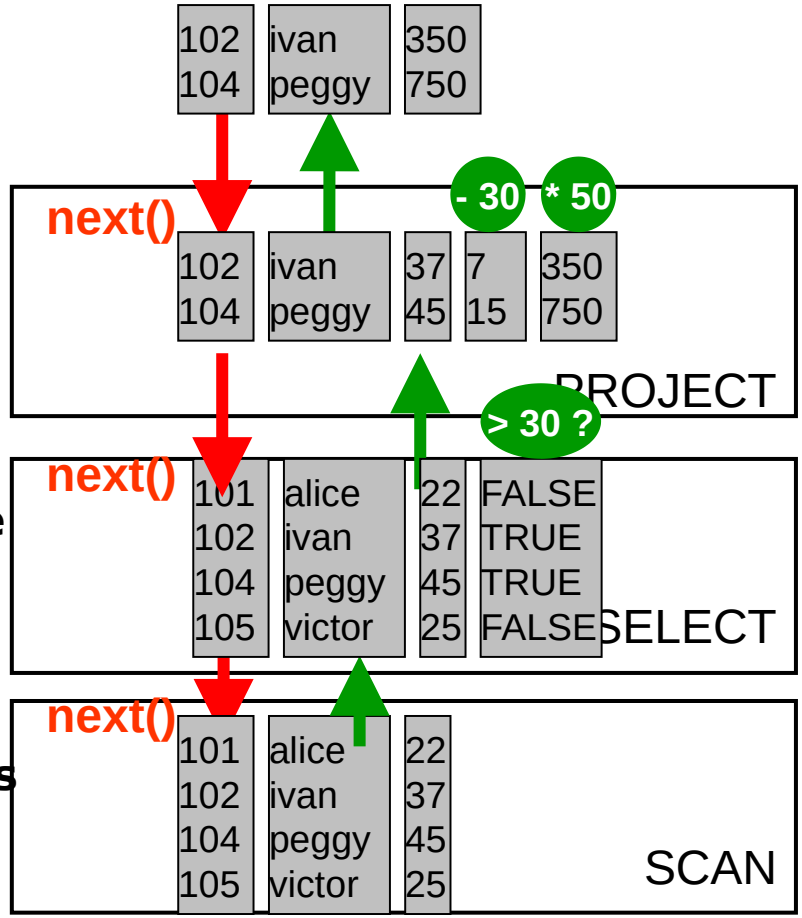
Observations:

next() called much less often ⇒ more time spent in primitives less in overhead

primitive calls process an array of values in a **loop**:

CPU Efficiency depends on "nice" code
- out-of-order execution
- few dependencies (control,data)
- compiler support

Compilers like simple loops over arrays
- loop-pipelining
- automatic SIMD



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

Observations:

next() called much less often ⇒ more time spent in primitives less in overhead

primitive calls process an array of values in a **loop**:

CPU Efficiency depends on "nice" co
- out-of-order execution
- few dependencies (control,data)
- compiler support

Compilers like simple loops over arra
- loop-pipelining
- automatic SIMD

> 30 ?

FALSE
TRUE
TRUE
FALSE

```
for(i=0; i<n; i++)
  res[i] = (col[i] > x)
```

- 30

7
15

```
for(i=0; i<n; i++)
  res[i] = (col[i] - x)
```

* 50

350
750

```
for(i=0; i<n; i++)
  res[i] = (col[i] * x)
```

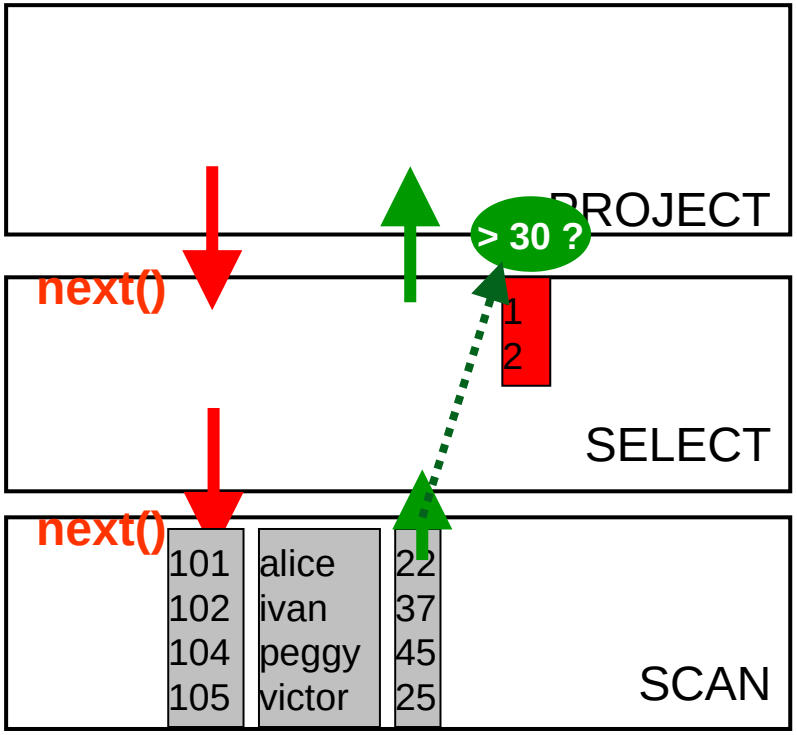
Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

Tricks being played:

- Late materialization

- Materialization avoidance using selection vectors



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

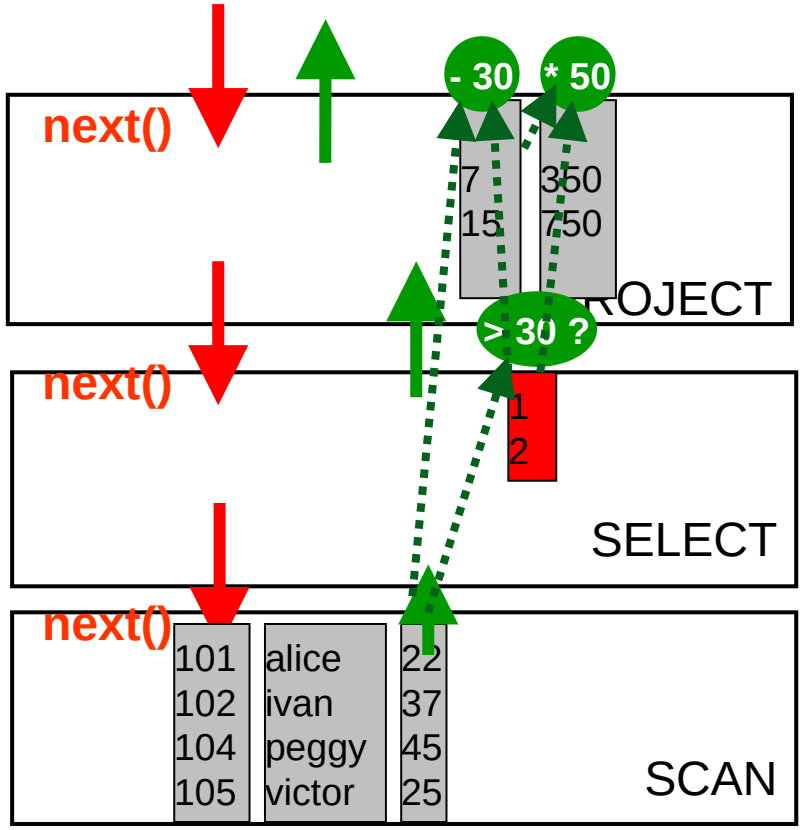
"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

```
map_mul_flt_val_flt_col(
    float *res,
    int*   sel,
    float  val,
    float *col, int n)

{
    for(int i=0; i<n; i++)
      res[i] = val * col[sel[i]];
}
```

selection vectors used to reduce vector copying

contain selected positions



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

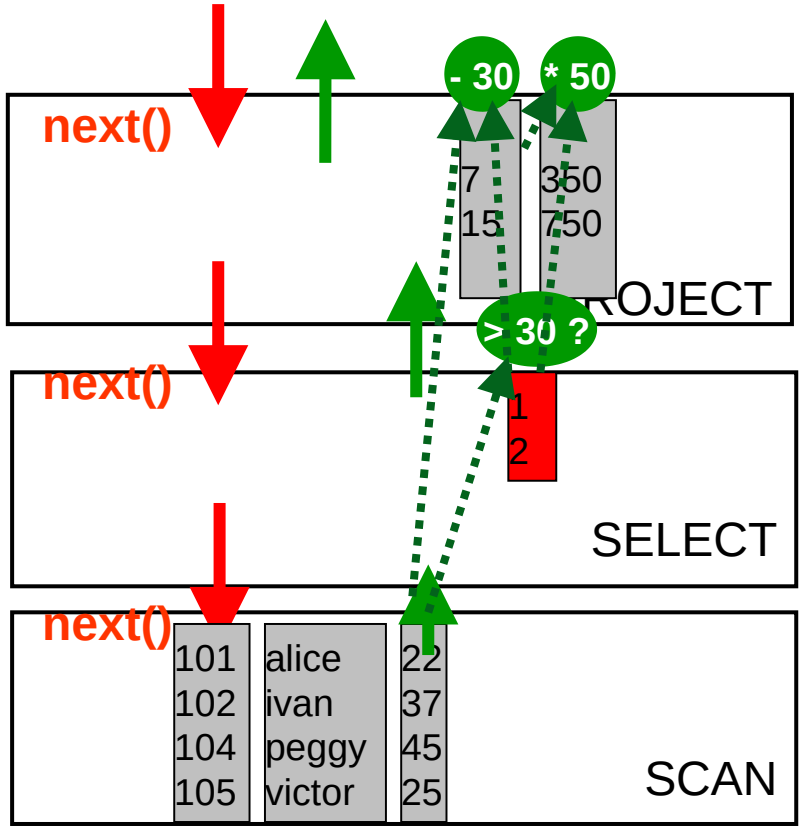
"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

```
map_mul_flt_val_flt_col(
    float *res,
    int*   sel,
    float  val,
    float *col, int n)

{
    for(int i=0; i<n; i++)
      res[i] = val * col[sel[i]];
}
```

selection vectors used to reduce vector copying

contain selected positions



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

VectorWise

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

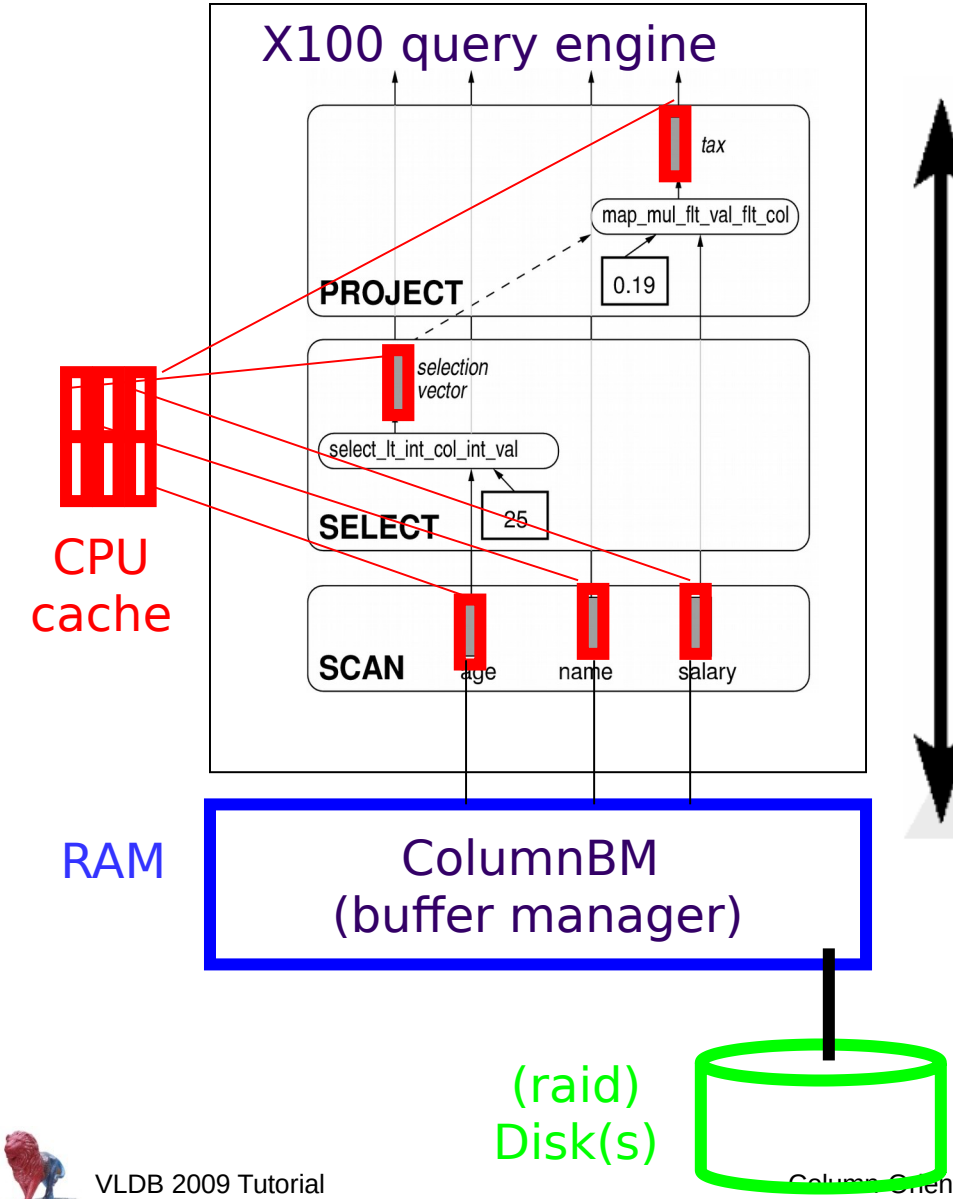
- Both efficiency
  - Vectorized primitives
- and scalability..
  - Pipelined query evaluation

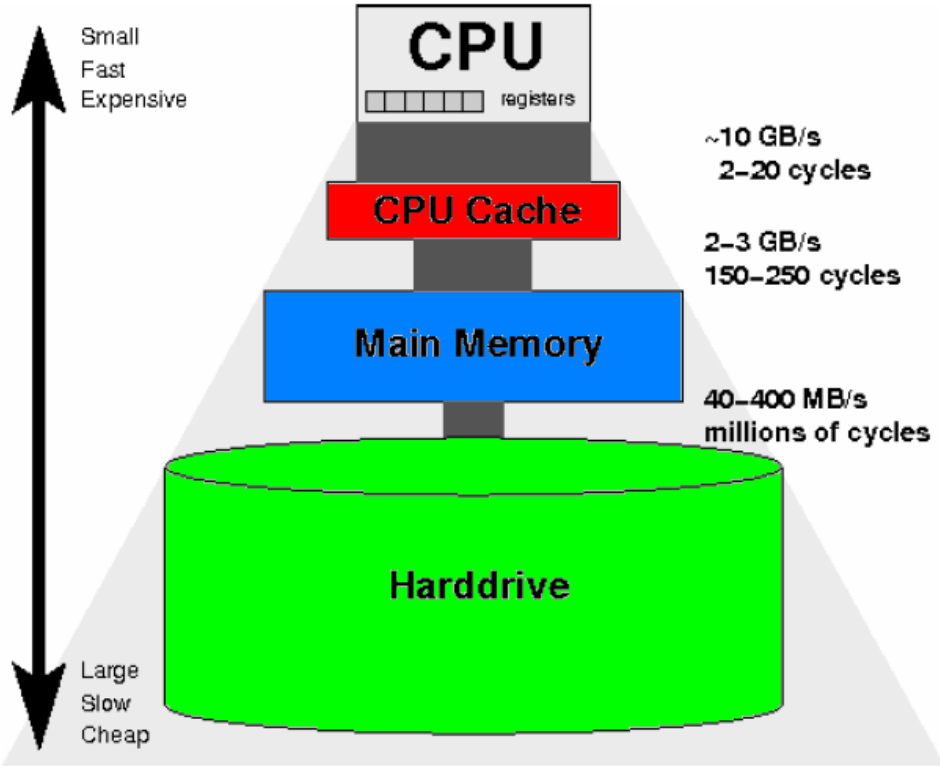
- C program: 0.2s
- MonetDB/X100: 0.6s
- MonetDB: 3.7s
- MySQL: 26.2s
- DBMS "X": 28.1s

Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

Memory Hierarchy

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

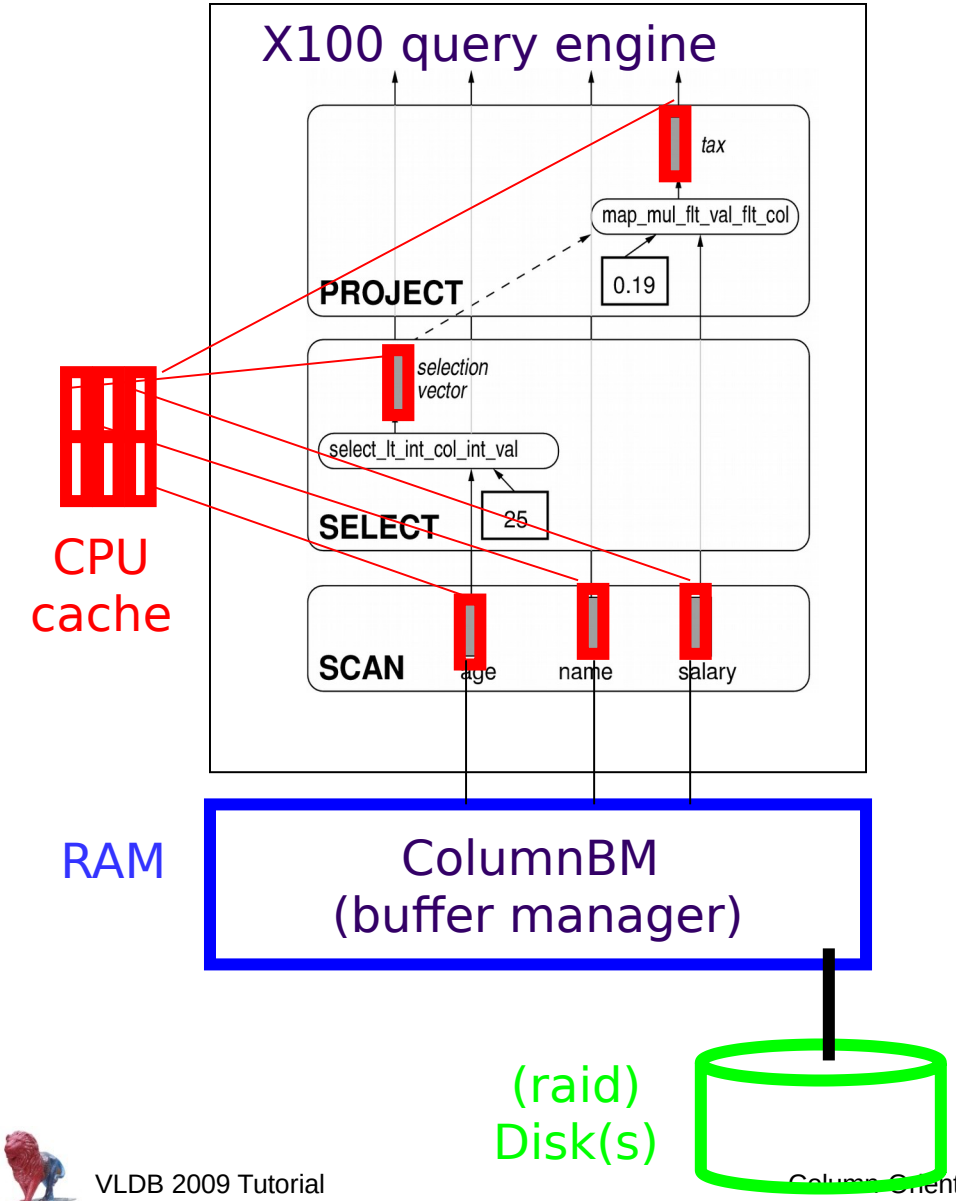




Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

Memory Hierarchy

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05



Vectors are only the in-cache representation

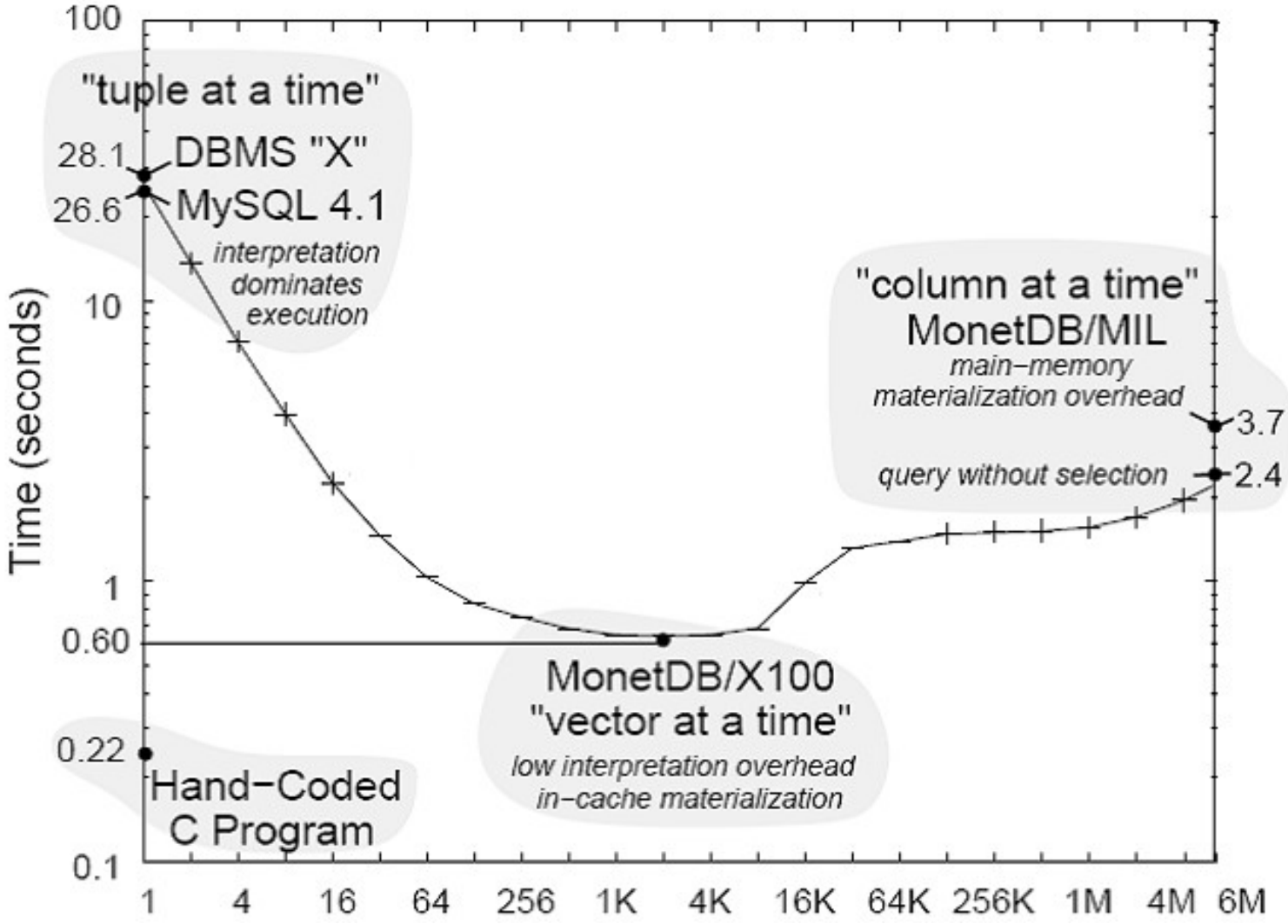
RAM & disk representation might actually be different

(vectorwise uses both PAX & DSM)

Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

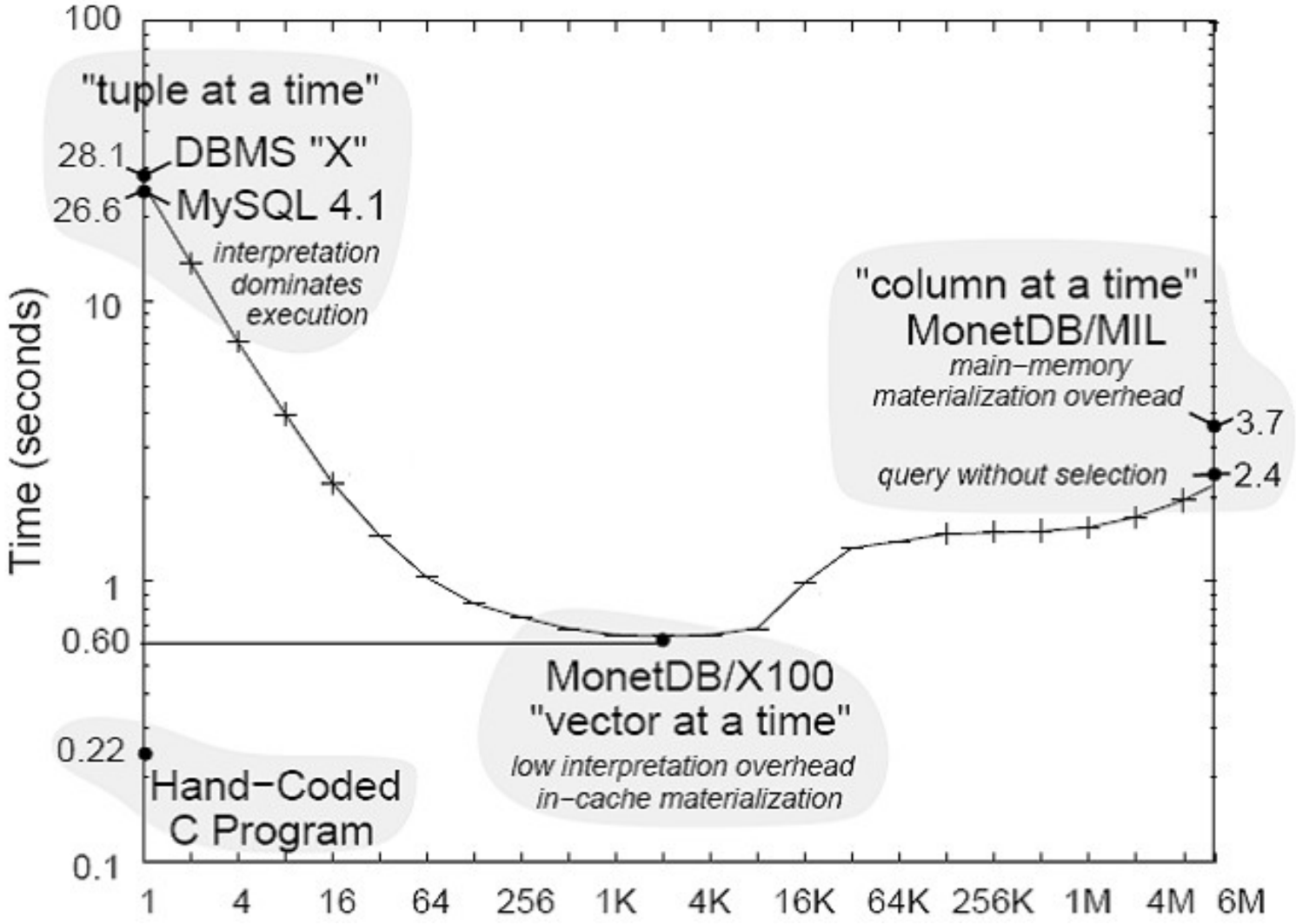
Varying the Vector size



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

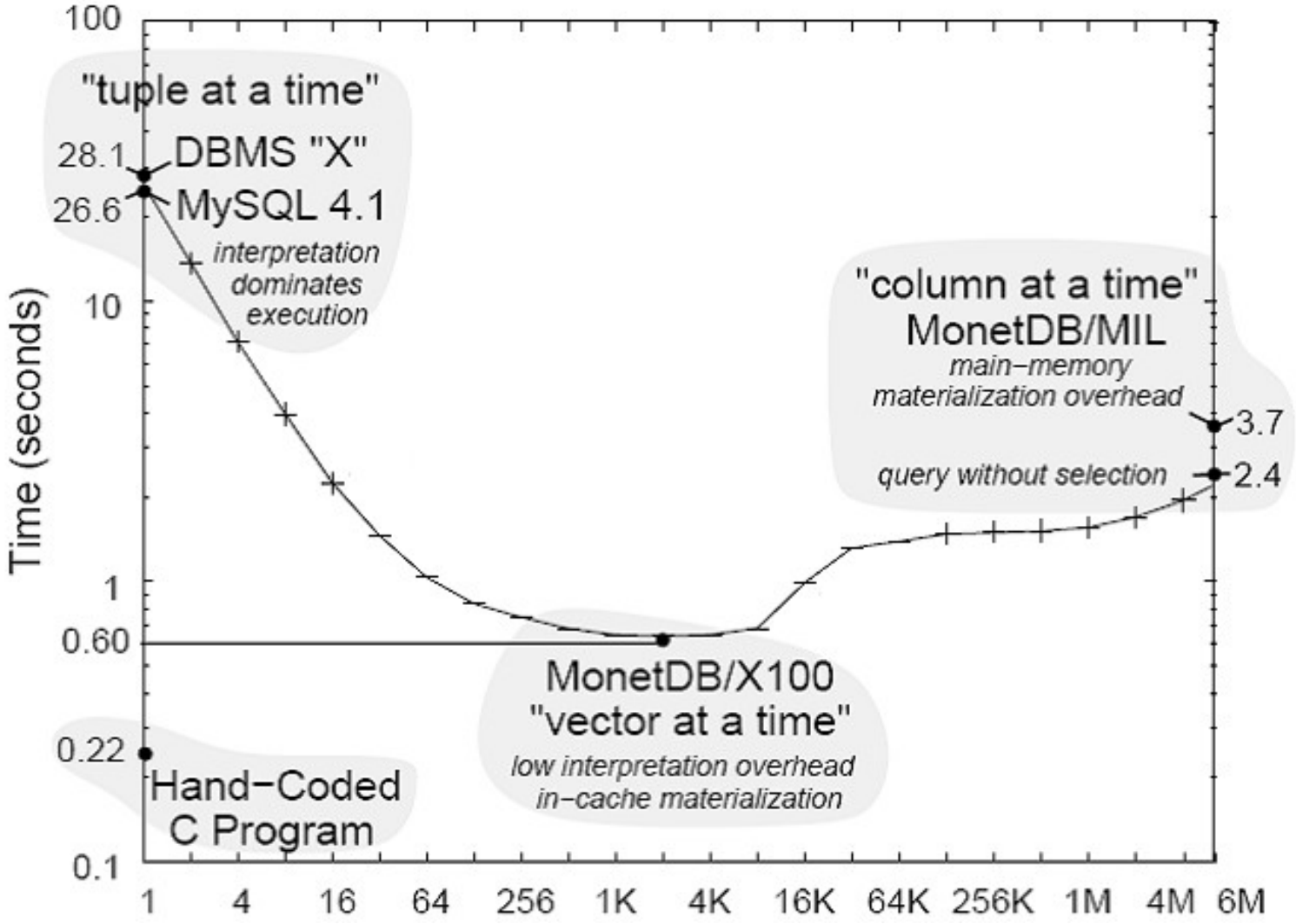
Varying the Vector size



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

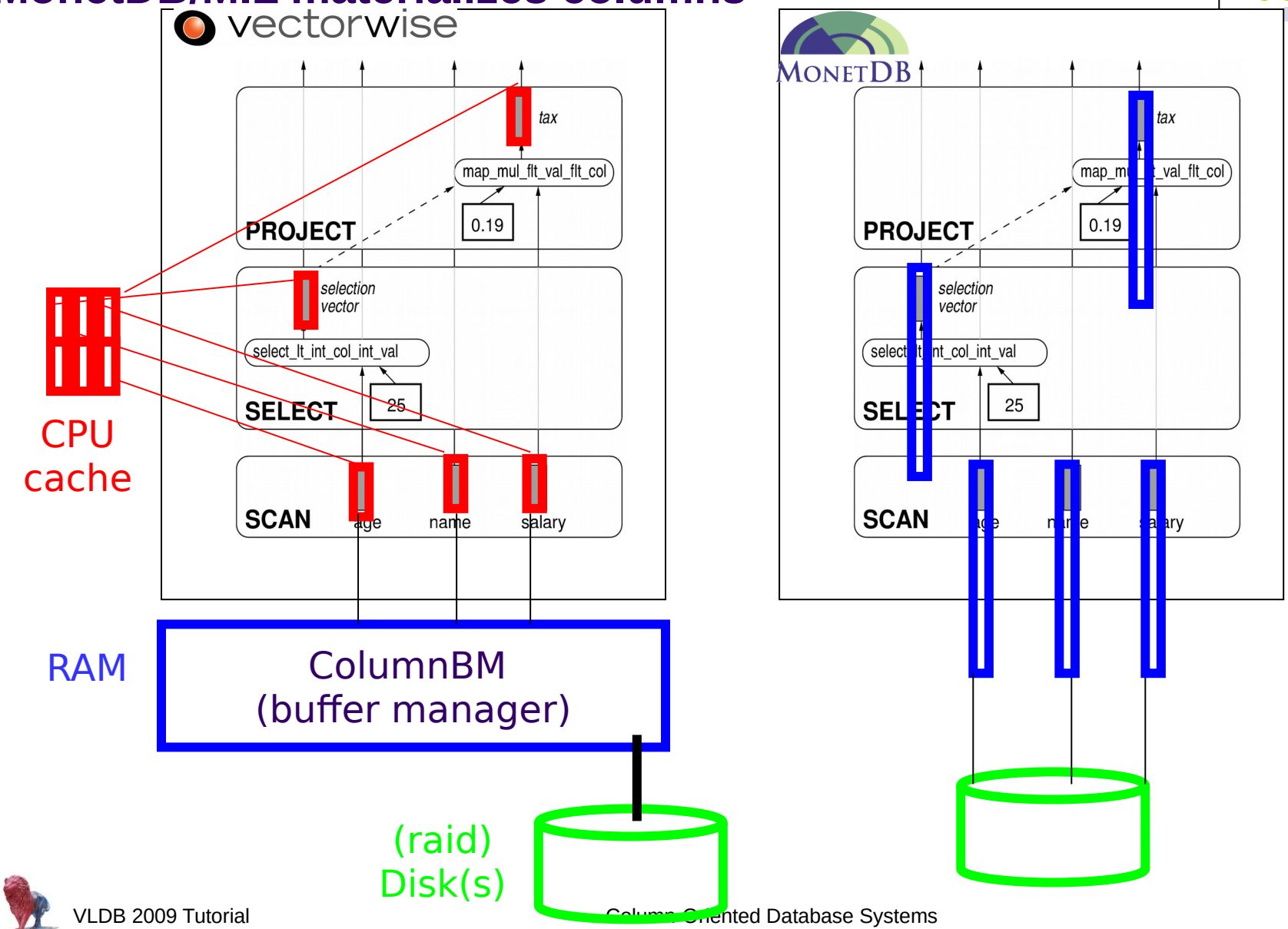
Varying the Vector size



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

MonetDB/MIL materializes columns



Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"MonetDB/X100: Hyper-Pipelining Query Execution " Boncz, Zukowski, Nes, CIDR'05

Benefits of Vectorized Processing

"Buffering Database Operations for Enhanced Instruction Cache Performance" Zhou, Ross, SIGMOD'04

"Block oriented processing of relational database operations in modern computer architectures" Padmanabhan, Malkemus, Agarwal, ICDE'01

- **100x less Function Calls**
  - iterator.next(), primitives
- **No Instruction Cache Misses**
  - High locality in the primitives
- **Less Data Cache Misses**
  - Cache-conscious data placement
- **No Tuple Navigation**
  - Primitives are record-oblivious, only see arrays
- **Vectorization allows algorithmic optimization**
  - Move activities out of the loop ("strength reduction")
- **Compiler-friendly function bodies**
  - Loop-pipelining, automatic SIMD

Re-use permitted when acknowledging the original © Stavros Harizopoulos, Daniel Abadi, Peter Boncz (2009)

"Balancing Vectorized Query Execution with Bandwidth Optimized Storage " Zukowski, CWI 2008

Vectorizing Relational Operators

- Project
- Select
  - Exploit selectivities, test buffer overflow
- Aggregation
  - Ordered, Hashed
- Sort
  - Radix-sort nicely vectorizes
- Join
  - Merge-join + Hash-join

Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask

Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, Peter Boncz

Query Engine Paradigms (1)

there are two state-of-the query processing paradigms:

- vectorized execution[1]
  used by Vectorwise, SQL Server (Apollo), DB2 BLU, Quickstep, ...
- data-centric code generation[2]
  used by HyPer, Peloton, Spark, ...

[1] MonetDB/X100: Hyper-Pipelining Query Execution, Boncz et al., CIDR 2005
[2] Efficiently Compiling Efficient Query Plans for Modern Hardware, Neumann, VLDB 2011

Query Engine Paradigms (2)

- both eliminate the overhead of Volcano-style engines and are efficient
- conceptually very different from each other:
  - pull vs. push model
  - vector-at-a-time vs. tuple-at-a-time processing
  - interpretation vs. compilation

Hash Join Using Vectorization

```
class HashJoin
  Primitives probeHash_, compareKeys_, buildGather_;
  ...
int HashJoin::next()
  ... //consume build side and create hash table
  int n = probe->next()//get tuples from probe side
  // *Interpretation*: compute hashes
  vec<int> hashes = probeHash_.eval(n)
  // find hash candidate matches for hashes
  vec<Entry*> candidates = ht.findCandidates(hashes)
  //matches: int references a position in hashes
  vec<Entry*, int> matches = {}
  // check candidates to find matches
  while (candidates.size())
    // *Interpretation*
    vec<bool> isEqual = compareKeys_.eval(n, candidates)
    hits, candidates = extractHits(isEqual, candidates)
    matches += hits
  //*Interpretation*: gather from hash table into
  //buffers for next operator
  buildGather_.eval(matches)
  return matches.size()
```

Hash Join Using Data-Centric Code Generation

```
query(...)
  // build hash table
  for (i = 0; i < S.size(); i++)
    ht.insert(<S.att1[i], S.att2[i]>, S.att3[i])
  // probe hash table
  for (i = 0; i < R.size(); i++)
    int k1 = R.att1[i]
    string* k2 = R.att2[i]
    int hash = hash(k1, k2)
    for(Entry* e = ht.find(hash); e; e = e->next)
      if(e->key1 == k1 && e->key2 == *k2)
        ... //code of parent operator
```

If one wants to build a new query engine today,
which paradigm should one use?

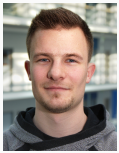
Vectorwise vs. HyPer?

actian Vectorwise

HyPer

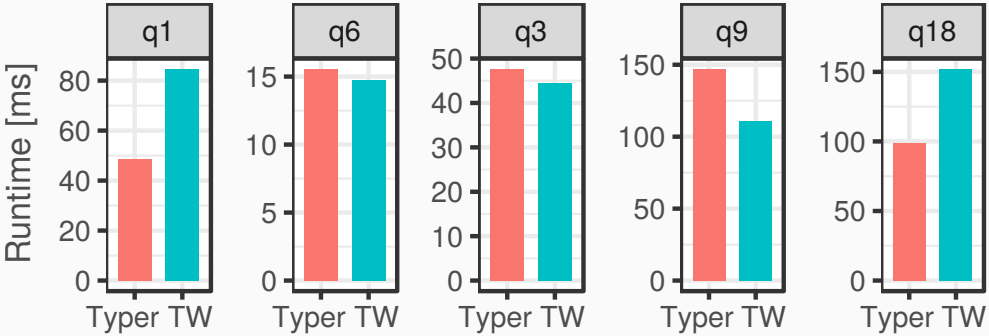
- different storage/compression schemes
- different parallelization frameworks
- different query processing algorithms and data structures
- different query optimizers
- ...

Tectorwise vs. Typer

Tectorwise  Typer

- same storage scheme (mmap-ed uncompressed columns)
- same parallelization framework (morsel-driven parallelization)
- same query processing algorithms and data structures (identical hash tables)
- same query optimizer (identical query plans)
- ...

TPC-H (SF=1, 1 thread, no SIMD)



Microarchitectural Analysis: TPC-H Q1

```
select   l_returnflag, l_linestatus,
         sum(l_quantity),
         sum(l_extendedprice),
         sum(l_extendedprice * (1 - l_discount)),
         sum(l_extendedprice * (1 - l_discount) * (1 + l_tax)),
         avg(l_quantity),
         avg(l_extendedprice),
         avg(l_discount),
         count(*)
from     lineitem
where    l_shipdate <= date '1998-12-01' - interval '90' day
group by l_returnflag, l_linestatus
order by l_returnflag, l_linestatus
```

	cycles	IPC	instr.	L1 miss	LLC miss	branch miss	mem. stall cycles
Typer	34	2.0	**68**	0.6	0.57	0.01	1.8
Tectorwise	59	2.8	**162**	2.0	0.57	0.03	5.2

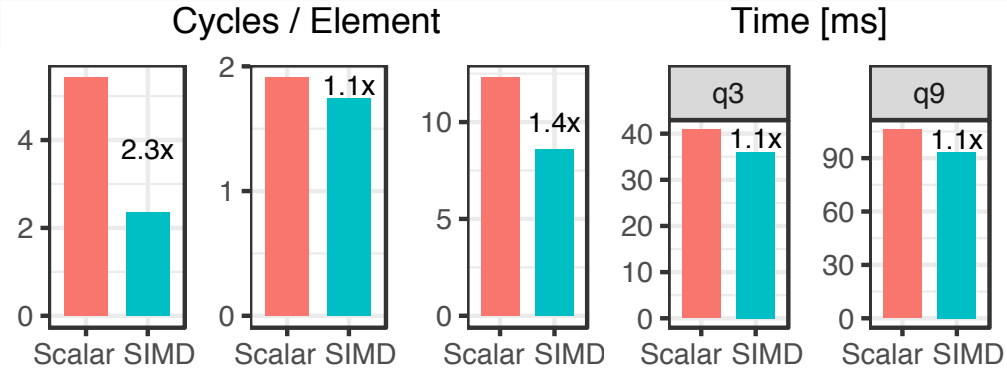
Microarchitectural Analysis: TPC-H Q9

```
select ...
from lineitem, orders, partsupp,
     supplier, part, nation
where s_suppkey = l_suppkey
and ps_suppkey = l_suppkey
and ps_partkey = l_partkey
and p_partkey = l_partkey
and o_orderkey = l_orderkey
and s_nationkey = n_nationkey
and p_name like '%green%'
```

	cycles	IPC	instr.	L1 miss	LLC miss	branch miss	mem. stall cycles
Typer	74	0.6	42	1.7	0.46	0.34	**42**
Tectorwise	56	1.3	76	2.1	0.47	0.39	**18**

Stall Cycles

What About SIMD? (SIMD Hashing in Tectorwise)



dense hashing gather sparse hashing

OLAP Throughput

$<$ *Computation:* compiled code can keep data in CPU registers

$>$ *Parallel data access:* vectorization is better at generating parallel cache misses

$=$ *SIMD:* easier to apply to vectorization, though gains are limited

$=$ *Parallelization:* both scale well

$=$ *Hardware platforms:* Skylake, Knights Landing, and AMD Ryzen are similar

$\Rightarrow$ overall performance similar on OLAP, no clear winner

Other Aspects Are Probably More Important

- < *OLTP:* compilation results in fast stored procedures
- < *language support:* compilation enables seamless integration of different languages
- > *compile time:* vectorization has no JIT-compilation overhead
- > *profiling*: runtime can easily be attributed to vectorized primitives
- > *(micro-)adaptivity*: vectorization allows primitives to be swapped mid-fligh

for optimal performance, one needs a hybrid engine that combines both approaches (Data Blocks[3] in HyPer, Relaxed Operator Fusion[4] in Peloton)

[3] Data Blocks: Hybrid OLTP and OLAP on Compressed Storage using both Vectorization and Compilation, Lang et al., SIGMOD 2016

[4] Relaxed Operator Fusion for In-Memory Databases: Making Compilation, Vectorization, and Prefetching Work Together At Last, Menon et al., VLDB 2018

Everything You Always Wanted to Know About Compiled and Vectorized Queries But Were Afraid to Ask

Timo Kersten, Viktor Leis, Alfons Kemper, Thomas Neumann, Andrew Pavlo, Peter Boncz

PVLDB 2018, Vol. 11, No. 13

source code: `https://github.com/TimoKersten/db-engine-paradigms`